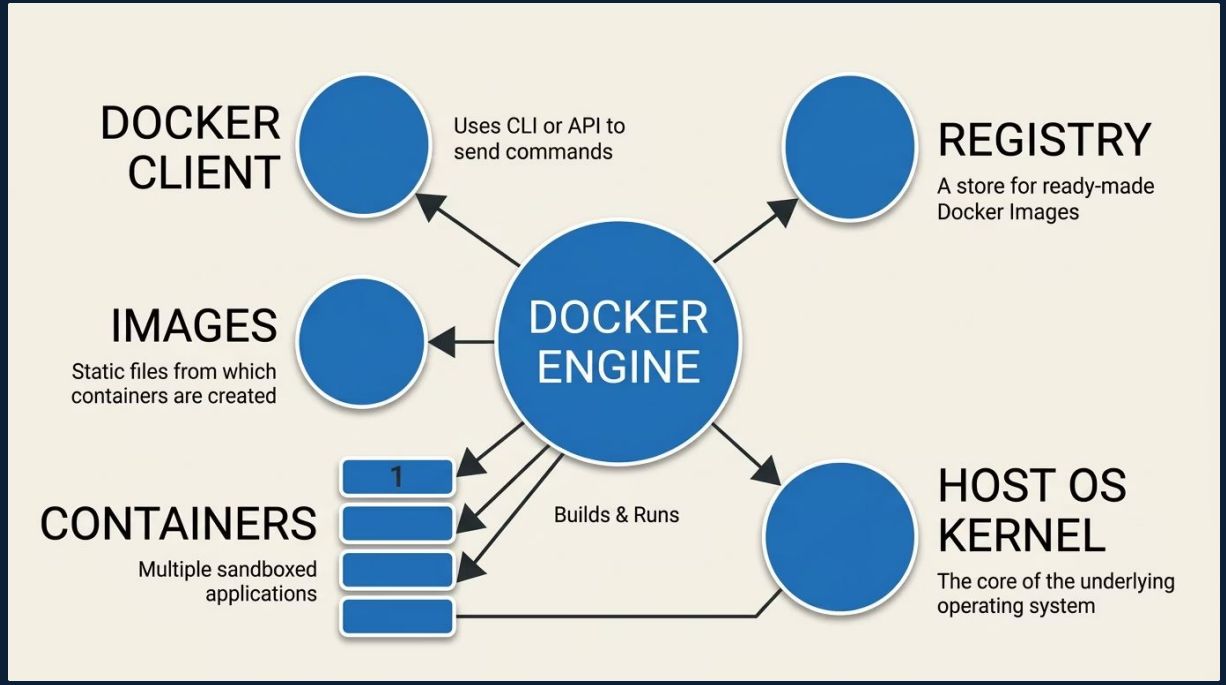
SLIDE 1: DOCKER — INTRODUCTION

Definition: Docker is an open-source platform for building, packaging, and running applications in isolated containers.

Core Components: Docker Engine, Images, Containers, Dockerfile, and Docker Hub.

Working: Source Code → Dockerfile → Image → Container → Application.

Purpose: Ensures consistent application environments across development, testing, and production.



Made with GAMMA

KEY ADVANTAGES OF DOCKER

Portability:

Runs applications across compatible environments.

Lightweight:

Shares the host OS kernel, reducing overhead compared with full virtual machines.

Speed:

Enables rapid container startup and deployment.

Consistency:

Packages applications with their dependencies.

Resource Efficiency:

Allows multiple containers to share host resources.

Scalability:

Supports replication and workload management.

Made with GAMMA

APPLICATIONS AND BENEFITS



Code → Build → Test → Registry → Deploy.

Cloud Computing: Deploys applications across cloud environments.

CI/CD: Supports automated software building, testing, and deployment.

Microservices: Packages individual services for independent deployment.

Development and Testing: Creates reproducible, isolated environments.

DevOps: Improves collaboration and standardizes software delivery.

Made with GAMMA

MAJOR CHALLENGES OF DOCKER

Security:	Vulnerable images, excessive privileges, and misconfigurations create risks.
Storage:	Requires persistent volumes and reliable backups.
Networking:	Needs proper port mapping, service discovery, and network configuration.
Monitoring:	Troubleshooting distributed containers can be complex.
Resource Management:	Uncontrolled CPU and memory usage can affect performance.
OS Compatibility:	Containers require a compatible host kernel environment.

Made with GAMMA

OPERATIONAL CHALLENGES

INTERCONNECTED LAYERS

APPLICATION IMAGES

RUNTIME

NETWORKING

STORAGE

SECURITY

MONITORING

- **Complexity:** Managing large container deployments requires additional tools.
- **Image Management:** Large or outdated images increase storage and security risks.
- **Dependency Conflicts:** Incorrect versions can cause deployment failures.
- **Configuration Management:** Secrets and environment variables require secure handling.
- **Learning Curve:** Effective container management requires technical expertise.

Docker simplifies packaging but does not eliminate infrastructure-management responsibilities.

Made with GAMMA

SOLUTIONS AND BEST PRACTICES

- Use trusted, updated, minimal images and scan for vulnerabilities.
- Apply least-privilege permissions and store secrets securely.
- Use Docker volumes for persistent data and backups.
- Configure networks, port access, and CPU/memory limits carefully.
- Optimize Dockerfiles and remove unnecessary dependencies.
- Monitor logs, health checks, and resource usage.
- Use Docker Compose for multi-container applications and Kubernetes when - when orchestration needs justify it.

ESSENTIAL DOCKER COMMANDS

```
docker pull nginx — Download an image.
docker images — List images.
docker run -d -p 8080:80 nginx — Run a container.
docker ps — List running containers.
docker logs <container_id> — View logs.
docker stop <container_id> — Stop a container.
```

Made with GAMMA

SLIDE 7 / COMPARATIVE ANALYSIS, CONCLUSION, AND KEY TAKEAWAYS

DOCKER VS VMS AND CONCLUSION

FEATURE	DOCKER CONTAINERS	VIRTUAL MACHINES
Architecture	Share host OS kernel	Run guest OS
Overhead	Generally lower	Generally higher
Startup	Usually faster	Usually slower
Isolation	Process-level isolation	Separate guest OS
Best suited for	Application deployment	Full OS environments

Conclusion

Docker simplifies application deployment through portability, consistency, and efficient resource utilization. However, security, storage, networking, and operational complexity require careful management.

Final Message:

Docker is a powerful containerization technology when combined with secure configuration and effective operational practices.

CLOSING STATEMENT

Thank You | Questions & Discussion.

Made with GAMMA